Axyl Fredrick

Dr. Erik Grimmelmann

CSC 44700 – Machine Learning

Application of NEAT in *Sonic the Hedgehog 2*

Overview:

Video games are mainly played by humans for their own learning, but it also provides an interesting application for machine learning. *Sonic the Hedgehog 2* for the Sega Genesis is a beloved video game, played by many generations of human beings across the world, and also played by various generations of the neural network covered in this paper. The NEAT model, or neuro-evolution of augmented topologies, is a very famed machine learning and neural network framework created by researchers Kenneth Stanley and Risto Miikkulainen in 2002. Unlike other neural networks, NEAT creates its own arbitrary weights, in contrast to those neural networks where human beings determine the actual weights being used. The structure of the network continues to change and update itself, therefore the strongest networks survive with the best gameplay; this makes NEAT a neat option for a machine learning model that can play a video game. The purpose of this paper is to convey the findings of test runs of a NEAT algorithm on different stages of the video game *Sonic the Hedgehog 2*, both the computational findings and the heights and hurdles faced throughout the project. Various Python libraries were used, including NEAT-Python, NumPy, OpenCV, GraphViz, MatPlotLib, and (most crucially for communication between the game and Python) OpenAI's Gym Retro library.

The reason why *Sonic the Hedgehog 2* was chosen was due to its accessibility, popularity, and the uniqueness of the game mechanics. First of all, *Sonic the Hedgehog 2* is easily obtainable as a ROM file from the original game cartridge, and the ROMs can be obtained through simple

and lawful means, unlike other retro video games. To play the game for this model, the Sega Genesis is emulated on one's computer using the emulator Genesis Plus GX, which comes with the Gym Retro library, and outputs from the model are fed as button presses to the emulated console from the Python script and using the Gym Retro library.

Sonic 2 is also a very popular game, meaning that resources on the game's physics, memory addresses, and other details regarding the game are widely accessible. The Sonic community has extensively documented the game and its innards and decompiled it, and emulation tools like BizHawk were used to find memory addresses for player variables such as score and time. In addition, the mechanics of *Sonic the Hedgehog 2* as a video game differ from its contemporaries, as it is a physics and momentum based platformer. Another game like *Snake* or *Pong* would have been easier to train, needing less outputs and to learn less, but the mechanics of *Sonic* games are special. Though other platform games of *Sonic 2*'s day such as *Super Mario World*, *Kirby's Adventure*, and *Super Metroid* also have varying terrain with slopes, *Sonic 2* is unique because of its intricate physics system and the game's fast pace. Sonic, the titular hedgehog and main character, is able to run up walls smoothly, and go through loops should he be fast enough to traverse them. Sonic can also run up a wall or slope and be unable to traverse it due to not having enough speed, but when Sonic rolls he is able to move faster. This mechanic makes it interesting for the neural network to traverse and learn from, but it is also a detriment for the actual training, namely in cases where Sonic repeatedly tries to traverse a hill and fails.

Sonic 2 has very simple controls and uses 5 buttons: the four D-pad inputs and one jump button (which is repeated on the A, B, and C buttons on the Sega Genesis controller; all these inputs are used to allow for frame-perfect jump presses). There are also four actions that can be done with these inputs: running (with a left or right input), jumping (with the A, B, or C buttons),

rolling (by pressing down while Sonic is in motion on the ground), and spin-dashing (by pressing down and one of the A, B, or C buttons while Sonic is at a standstill). Other games of this era were more complex with regard to inputs and actions, but these actions all have a high skill ceiling due to the terrain the stages in the game provide, which make it more conducive to training and rewarding a high performing model with its simplicity.

The Python Code:

Sonic 2's game input was retrieved using a Python library called Gym Retro, which was developed by OpenAI for use in older video games to make them suitable environments for reinforcement learning using Gym. This library turns the audiovisual input of the game into a Python environment for easy testing and training using its own API to communicate between the learning algorithm and environments, and acts as an interface for the emulator to perform the buttons pressed that the neural network outputs provide. It would have been possible to feed inputs to the emulator by itself, but it was easier to feed these inputs using a pre-existing library that has been tailored for the use of libraries such as the NEAT-Python library. Gym and Gym Retro are deprecated libraries however, so Python 3.8 was used for this project as there was trouble getting the more recent and currently maintained forks of those projects (Gymnasium and Stable-Retro) to work.

For each genome in the genomes, the action space is randomly sampled from the 12 possible outputs on the Genesis controller, so all the inputs that the species makes are determined. Every frame that the game is running, the environment is updated in Gym, and the observation space is pre-processed by OpenCV. The environment is then updated by actions from

the neural network through the Gym API, and the env.step function is called from Gym; the observation, reward, done flag, and miscellaneous data are unpacked using tuple unpacking.

NEAT-Python was another library used, and it contains the method to create the recurrent network that was used, along with the activation method. A recurrent neural network was used because recurrent networks remember the past inputs, due to being used to remember sequential data. Such a network is dynamic and continues to pass information into each time step, and parameters are passed over time steps (Zhang, 2024, 9.1). In any video game, the order of inputs matter; therefore the only way for the neural network to be able to learn and make progress through the level is through having memory of past inputs. The recurrent neural network used for the model used a sigmoid activation function, because the sigmoid function is nonlinear and is therefore good for adaptation. Many to many modeling was used for the project, as the game screen cannot be passed by itself and many outputs are needed to know what buttons need to be pressed to move Sonic. The other option that could have been used and that was also included with the NEAT-Python library was feed-forward networks, which do not have the characteristic of memory that recurrent networks do, as information flows in a single direction (as per the name "feed-forward").

OpenCV was used to downsample the game's visual output, such that there would be less inputs than sending each individual pixel. Though it would have been possible to send 71,680 pixels (as the Genesis screen for *Sonic 2*'s single player mode on NTSC regions has a resolution of 320 pixels by 224 pixels) as 71,680 inputs for the neural network, not only would that have been unhelpful to nudge the outputs in any good direction for a playable run, but it also would have been a computational nightmare. Only 1,120 inputs are used, because the Genesis screen is cut into eighths on each end, giving measurements in chunks of 40 pixels by 28 pixels. This size

is around the size of Sonic's sprite in-game, which also means the chunks are neither too large to get any information, nor too small to make distinctions between any object on screen. These chunks are put into 1-dimensional grayscale using the OpenCV cvtColor function, with brightness unadjusted and no scaling done, and are what the NEAT model sees and trains on for inputs. Figure 1 (in appendix) is a demonstration of what the model actually looks at taken from a test run of the stage Chemical Plant Zone Act 2.

GraphViz and MatPlotLib were both used for the visualization of the model. The NEAT-Python library uses GraphViz to generate plots using its own visualize method included, and this was used within the code to draw what the neural net itself looks like, as well as plot the speciation and best fitness. These were included in a visualize Python file in the NEAT-Python directory.

Retrieval of Data:

Data was collected with all the aforementioned libraries using a NEAT (NeuroEvolution of Augmenting Topologies) model. The primary feature of NEAT and reason why it was selected for use in this project is its speciation. NEAT is a genetic algorithm, where a population of genomes is maintained and preserved. These genomes contain connection genes, which show the connection between two genes, and the innovation number, which finds corresponding genes during reproduction (Stanley, 2002, p. 1757-1759). Genomes go through crossover, a process in which two genes are combined and mate to produce a new gene, and the innovation number is NEAT's innovation that allows for such breeding to occur.

The main benefit of this reproduction, just as in similar forms of reproduction in the natural world, is the diversity of possibilities for the next generation of genes. This diversity is

conducive to video games like *Sonic 2*, where some generations perform actions like jumping at different times, and some actions are better than others. The best actions are reinforced and remain, whereas the actions that do not allow for progress in the stage (e.g. Sonic running to the left, which is away from the goal on the right) are unable to survive. The generations where Sonic survives without getting hit by spikes or enemies, goes to the right, and makes jumps are the ones that are most successful, and the model sees these genes and improves.

The incentive for these models to improve is fitness, which increases if an action is made that is closer to the goal of reaching the end of the level. Many fitness criteria were used, including rings, score (which increases after Sonic defeats enemies called Badniks, along with the score bonus at the end of the level), and how much forward progress Sonic makes in the stage (that is, his x position on the stage). The ring reward was 1 for each ring collected, the score reward was 1 for each point added to the score, and the forward progress reward was 1 for each increase in Sonic's x position; the forward progress reward was set 5 times the others to incentivize level completion. An additional bonus reward if Sonic beat the level was given worth 500,000, instantly surpassing any of the other targets on their own and specially incentivizing those models that have made it to the end of the stage. The forward progress reward was the most as it is important for Sonic to progress through the level, but the other measures like rings were still used to give more goals for the neural network and reward good play. There is one problem with rings as a criterion that was not addressed and did not have a significant impact on the performance of the model in the runs performed, but may be an issue–rings are recollectable after Sonic is hit by an enemy. This means that in the worst case scenario, bad gameplay could be rewarded due to the recollection of rings, though it is always more fruitful not to get hit due to the score reward balancing it out.

Another potential issue with this criterion is the score and the existence of exploits where the player could obtain a theoretically infinite number of points (were it not for the time limit)–though this is not a problem that occurs in *Sonic 2*; however such an exploit does exist in *Sonic 3* in the stage Launch Base Zone Act 1. However, the glitch involves the player entering into a Spin Dash state, which is one of the less likely states for the neural network to run into, given that the model rewards forward movement and the right directional button is faster and more likely. Therefore, the model being impeded by this glitch is possible in theory, but not a likely outcome. In addition, the reward for Sonic completing the level makes it so that the generations that finish the level survive.

Even the simple rightward movement criterion has its own flaw. When Sonic runs up terrain and is unable to scale it, he runs back down in the opposite direction of where he is trying to go. While this makes for an interesting game mechanic, and given time it would have been interesting to see how the model tackles these hills, what happens on most runs is that Sonic runs back down the hill repeatedly and the model never actually makes it up the hill (Figure 2 in appendix shows what this situation looks like). What is especially disheartening about Figure 2 is that there is a spring to the left that Sonic could very well hit, but because of the incentive for him to go right, the model never considers the possibility of turning around in that instance, though with enough runs anything is possible. The stopping condition is when fitness no longer increases after 200 idle frames, which occurs when Sonic reaches a standstill. Despite this, there is no way to know if Sonic is in this feedback loop by the metrics used for this model, as Sonic is constantly moving. One would have to wait for the timer to run out after 10 minutes in order for a new generation to begin another run. This problem is not a neural network problem, but a *Sonic 2* problem; it could perhaps be fixed by cutting the generation short when the timer reaches 5

minutes as opposed to the in-game time of 10 minutes, either through cutting the generation short after a certain number of frames have passed or through editing the *Sonic 2* ROM and adjusting the maximum time of the level to 5 minutes.

In spite of all these impediments, having more than one criterion rewards models that play well rather than simply how far Sonic goes in the level. Sonic is able to make it to the end of the stage without collecting any rings or defeating any enemies, but truly skillful play is more than reaching the goalpost. Sonic games also reward player exploration by placing in-game items such as 10-ring monitors, invincibility monitors, and shield monitors in hard to reach places or places that diverge from the main path. *Sonic 2* in particular has many paths that a player could traverse through and cross between, unlike in other platform games like *Super Mario Bros.* where there is usually only one main path to traverse. If one generation finds items along the level path or deviates from the main path of the level for exploration on its own, the genomes from that generation of good play would be able to pass onto the next generation. A generation that makes it to the top path of the level, which is usually the best path in most *Sonic 2* acts, is a generation that survives, as it knows how to navigate skillfully. The ability to take different paths and take risks through NEAT's innovation process is what made using NEAT so attractive for this project, and it worked seamlessly with the level design paradigm of *Sonic* games on the best runs.

Data and Other Findings:

Three test runs where data was successfully recorded were performed: two with Sonic in the stage Emerald Hill Zone Act 1, and another in Chemical Plant Zone Act 2. These test runs were performed on populations of 35 species for 2-4 generations, and a survival threshold of 0.2,

along with a maximum stagnation of 2 generations. For stagnation, one species was elite, and the minimum species size was 0, allowing species to fully die out.

The model was tailored for rapid change, but it turns out that throughout the various test runs, the population remained at the initial threshold and the speciation did not change. The species themselves changed, but there did not seem to be a drastic change in the number of species that were there, even on some test runs where the stagnation was only for one generation (too weak to make any call, which is why it was not used). This is because the species were able to replace each other quickly and mutation occurred rapidly as the connections continued to change with each generation.

The fitness threshold for all the runs was 100,000. The first two runs managed to reach a fitness of above 150,000, and the second reached a final fitness of 115,000. All the models met this threshold after the generations for which they were run. For the Emerald Hill runs, there was a slump in the best and average fitness after a few generations, showing some stagnation occurring, but in the end the fitness goals were met. For the Chemical Plant run, it took only 2 generations to exceed the fitness goal, and no slump was seen, possibly because there were so few generations. One example of these test runs showing this slump is shown in Figure 3 (in appendix).

Unfortunately, in all the test runs performed, Sonic was not able to reach the end of any individual level due to the length of time it takes for the model to run and go through all of its generations. Even a modest run with a speciation of 30 among 10 generations would take 10 minutes in the worst case scenario for a level, resulting in 3,000 minutes or 50 hours of nonstop gameplay. While not resource-intensive, given that a modern computer is hundreds of thousands of times more powerful than a Sega Genesis, it is certainly time intensive. The emulator that

Gym Retro used for Genesis emulation has a fast-forward feature, but it seems that neither Gym Retro nor any of its forks have a way to access it. Though the worst case scenario for a level will not always occur, the 10 minute cases can occur frequently should Sonic run into certain terrain, as seen in Figure 2 (in appendix) during one of these runs. Due to this caveat, the evolution was cut short in order to be able to provide any data at all; had the models continued past the few generations they did, at least one of these species would have been able to finish a level, but the theoretical worst case alone rendered it impractical to perform. Raising the fitness threshold to 500,000 to make training stop only when the level is beat would have been too high of a barrier to cross and many more hours of idle training.

Some actions that Sonic took under the model were more frequently occurring than others. What ended up being surprisingly rare was the Spin Dash action, as Sonic had to stay at a standstill, and down on the D-pad and one of the jump buttons needed to be pressed. Sometimes the model will begin to charge a Spin Dash but never move, as the down D-pad press output needed to be stopped for Sonic to move. It was much more frequent that Sonic would go to the left in the level at the very beginning or make no moves at all than for a Spin Dash to occur. It could be due to how many inputs the Spin Dash requires. The Spin Dash revolutions that occur on each press of one of the jump buttons before Sonic charges off has its own counter, and it would have been possible to reinforce the model to Spin Dash by increasing the fitness with each revolution, but Sonic has to stop to perform a Spin Dash. Juggling this and the reinforcement of going forward is a difficult task, and would be an interesting hypothesis to test going forward. The other actions that Sonic could perform in game, such as rolling, running, and jumping were much more common, and the models picked up on pressing right and making jumps at the same time very quickly.

Overall, the model was able to reach the fitness goals given, not by beating the levels, but by being able to make a certain amount of horizontal progress within the level with a copious amount of rings collected and enough Badniks defeated. The bar was lowered from level completion to good level performance at the beginning, so that the model would have an attainable and timely fitness threshold to clear.

Conclusion:

A neural network was trained on *Sonic the Hedgehog 2* using the NEAT algorithm, managing to surpass the fitness goal of 100,000 on all 3 runs. The model took in 1,120 inputs from the game's visual data and ran them through the network to randomly sample outputs to the emulator that played *Sonic 2*.

Sonic 2 was chosen due to its physics mechanics and high speed gameplay, which provided an interesting experience for the model to learn from. The NEAT model used allowed for mutation and for Sonic to be able to traverse different paths and learn from the best. The multilayered level design of *Sonic 2* made it conducive for NEAT especially due to different simulations undergoing different routes.

The major feature that made *Sonic 2* special in this case and made it an area of interest–which was its physics system–was the Achilles' heel of the NEAT model. Sonic failed to traverse certain slopes and there did not seem to be a good way to circumvent this caveat. This led to data collection becoming arduous and time consuming and led to a reduction of the overall project goal from getting Sonic to beat a level to getting Sonic to clear the smaller goal of a certain fitness threshold.

Only three runs have had data retrieved, and it was hard to get data from more stages than the two that were tested. *Sonic 2* has 20 acts, and the more challenging zones such as Metropolis Zone provide new challenges and many deaths. It would have been harder to get Sonic to clear different fitness thresholds, but one test run was done without data on Metropolis Zone Act 3. That run was never completed as the fitness threshold was far off and very hard to obtain (the level is very difficult even for humans to complete; many a child has broken a controller over it), but in the future more test runs could be done.

In addition, other characters such as Knuckles (Tails doesn't have any of his own unique mechanics in the original Genesis *Sonic 2*), along with other *Sonic* games would be interesting avenues to explore. Perhaps a more linear game like *Streets of Rage* would be a better option for model training due to the pitfalls faced here with terrain.

Works Cited:

Zhang, A., Lipton, Z., Li, M., & Smola, A. (2024). *Dive into Deep Learning*. 9. Recurrent Neural Networks - Dive into Deep Learning 1.0.3 documentation. https://www.d2l.ai/chapter_recurrent-neural-networks/

Stanley, K. O., & Miikkulainen, R. (2002). Efficient evolution of neural network topologies. Proceedings of the 2002 Congress on Evolutionary Computation. CEC'02 (Cat. No.02TH8600), 2, 1757–1762. https://doi.org/10.1109/cec.2002.1004508

\

Appendix:

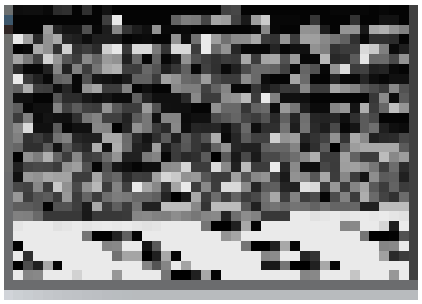

Figure 1: OpenCV output for a test run of Chemical Plant Zone.

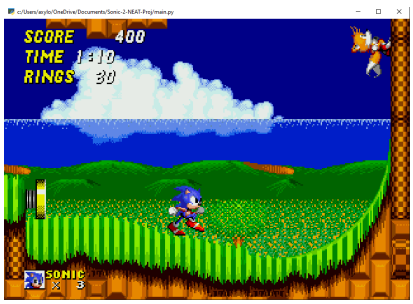

Figure 2: Sonic and Miles "Tails" Prower attempting to traverse a hill on one test run, not understanding that there is a spring to the left of them.

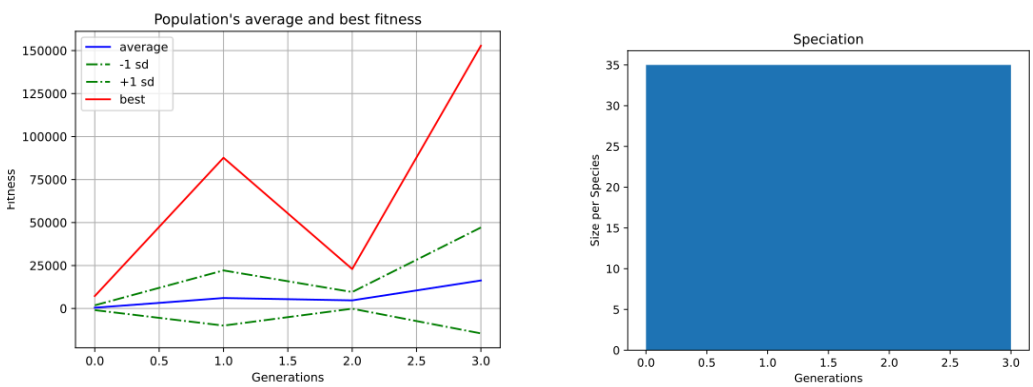

Figure 3: Sample data from one of the test runs performed in Emerald Hill Zone Act 1.